WEEK 9: SALES AND CUSTOMER ANALYTICS SYSTEM

Scenario:

The Art Workshop wants to analyze its sales data to understand customer purchasing behavior, product performance,and revenue trends. The management needs reports to identify top customers,best-selling products, and profitable categories.

1.Apply SQL Aggregate Functions:

SQL Aggregate functions perform calculations on a set of multiple records and return a single summarized value or a result for each group.

COUNT() – The COUNT() function counts the number of records in a table. SUM() – The SUM() function calculates the total of a numeric column. AVG() – The AVG() function calculates the average values of a numeric column. MAX() – The MAX() function returns the highest value in a column. MIN() – The MIN() function returns the lowest value in a column.

2.Generate Total Sales Reports:

Tasks: Write SQL queries to calculate:

1..Total number of orders placed

```
SQL> SELECT COUNT(*) AS ORDER_ID FROM ORDERS;

  ORDER_ID
----------
        15
```

2. Total revenue generated from sales

```
SQL> SELECT SUM(TOTAL_AMOUNT) FROM ORDERS;

SUM(TOTAL_AMOUNT)
-----------------
            42350
```

3.Average order value

```
SQL> SELECT AVG(TOTAL_AMOUNT) FROM ORDERS;

AVG(TOTAL_AMOUNT)
-----------------
       2823.33333
```

Average order value of each order at a time

```
SQL> SELECT ORDER_ID,AVG(TOTAL_AMOUNT) FROM ORDERS GROUP BY ORDER_ID;

ORDER_I AVG(TOTAL_AMOUNT)
------- -----------------
O001                 2500
O002                 1800
O003                 3200
O004                 1500
O005                 4200
O006                 2750
O007                 3600
O008                 1200
O009                 5100
O010                 2300
O011                 1900

ORDER_I AVG(TOTAL_AMOUNT)
------- -----------------
O012                 3400
O013                 2800
O014                 4500
O015                 1600

15 rows selected.
```

4.Highest and lowest order amount

```
SQL> SELECT MIN(TOTAL_AMOUNT) FROM ORDERS;

MIN(TOTAL_AMOUNT)
-----------------
             1200
```

```
SQL> SELECT MAX(TOTAL_AMOUNT) FROM ORDERS;

MAX(TOTAL_AMOUNT)
-----------------
             5100
```

```
SQL> SELECT MIN(TOTAL_AMOUNT) AS LOWEST_ORDERS FROM ORDERS;

LOWEST_ORDERS
-------------
         1200

SQL> SELECT MAX(TOTAL_AMOUNT) AS HIGHEST_ORDERS FROM ORDERS;

HIGHEST_ORDERS
--------------
          5100
```

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES FROM ORDERS WHERE ORDER_DATE BETWEEN '01-SEP-26' AND '09-SEP-26
';

TOTAL_SALES
-----------
      25850
```

5. Total sales for a specific period

```
SQL> SELECT COUNT(ORDER_ID) AS TOTAL_ORDERS, SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE, AVG(TOTAL_AMOUNT) AS AVG_OR
DER_VALUE FROM ORDERS;

TOTAL_ORDERS TOTAL_REVENUE AVG_ORDER_VALUE
------------ ------------- ---------------
          15         42350      2823.33333
```

3.Customer Purchase Analysis:

1. Total number of orders placed by each customer.

```
SQL> SELECT COUNT(*), CUSTOMER_ID FROM ORDERS GROUP BY CUSTOMER_ID;

  COUNT(*) CUSTOMER_ID
---------- -----------
         3           1
         1           6
         2           2
         2           4
         1           5
         1           8
         2           3
         1           7
         1           9
         1          10

10 rows selected.
```

2.Total amount spent by each customer.

```
SQL> SELECT CUSTOMER_ID,SUM(TOTAL_AMOUNT) FROM ORDERS GROUP BY CUSTOMER_ID;

CUSTOMER_ID SUM(TOTAL_AMOUNT)
----------- -----------------
          1             10800
          6              1200
          2              4550
          4              5800
          5              3600
          8              1900
          3              4900
          7              2300
          9              2800
         10              4500

10 rows selected.
```

3.Average spending per customer.

```
SQL> SELECT CUSTOMER_ID, AVG(TOTAL_AMOUNT) FROM ORDERS GROUP BY CUSTOMER_ID;

CUSTOMER_ID AVG(TOTAL_AMOUNT)
----------- -----------------
          1              3600
          6              1200
          2              2275
          4              2900
          5              3600
          8              1900
          3              2450
          7              2300
          9              2800
         10              4500

10 rows selected.
```

4. Customers with maximum purchase amount.

```
SQL> SELECT CUSTOMER_ID, TOTAL_AMOUNT AS MAX_PURCHASE_AMOUNT FROM ORDERS WHERE TOTAL_AMOUNT=(SELECT MAX(TOTA
L_AMOUNT) FROM ORDERS);

CUSTOMER_ID MAX_PURCHASE_AMOUNT
----------- ------------------
          1               5100
```

5.Customers with fewer purchases.

```
SQL> SELECT CUSTOMER_ID,TOTAL_AMOUNT AS MIN_PURCHASE_AMOUNT FROM ORDERS WHERE TOTAL_AMOUNT=(SELECT MIN(TOTAL
_AMOUNT) FROM ORDERS);

CUSTOMER_ID MIN_PURCHASE_AMOUNT
----------- ------------------
          6               1200
```

4.Find Top Customers Based on Purchase Amount:

1.Top 5 customers based on total spending

```
SQL> SELECT CUSTOMER_ID, SUM(TOTAL_AMOUNT) FROM ORDERS G
ROUP BY CUSTOMER_ID ORDER BY SUM(TOTAL_AMOUNT) DESC;

CUSTOMER_ID SUM(TOTAL_AMOUNT)
----------- -----------------
          1             10800
          4              5800
          3              4900
          2              4550
         10              4500
          5              3600
          9              2800
          7              2300
          8              1900
          6              1200

10 rows selected.
```

2. Customers with maximum number of orders

```
SQL> SELECT COUNT(*) AS NO_OF_ORDERS, CUSTOMER_ID
  2  FROM ORDERS
  3  GROUP BY CUSTOMER_ID
  4  ORDER BY NO_OF_ORDERS DESC;

NO_OF_ORDERS CUSTOMER_ID
------------ -----------
           3           1
           2           4
           2           3
           2           2
           1           9
           1           7
           1           8
           1          10
           1           6
           1           5

10 rows selected.
```

3. Frequent customers

```
SQL> SELECT CUSTOMER_ID,COUNT(*) AS NO_OF_ORDERS FROM OR
DERS
  2  GROUP BY CUSTOMER_ID ORDER BY NO_OF_ORDERS DESC;

CUSTOMER_ID NO_OF_ORDERS
----------- ------------
          1            3
          4            2
          3            2
          2            2
          9            1
          7            1
          8            1
         10            1
          6            1
          5            1

10 rows selected.
```

4. High-value customers

```
SQL> SELECT CUSTOMER_ID,SUM(TOTAL_AMOUNT) AS TOTAL_SPEND
ING FROM ORDERS
  2  GROUP BY CUSTOMER_ID
  3  ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_ID TOTAL_SPENDING
----------- --------------
          1          10800
          4           5800
          3           4900
          2           4550
         10           4500
          5           3600
          9           2800
          7           2300
          8           1900
          6           1200

10 rows selected.
```

5. Identify Best-Selling Products:

1.Products with maximum sales quantity

```
SQL> SELECT PRODUCT_NAME,STOCK_QUANTITY FROM PRODUCT ORD
ER BY STOCK_QUANTITY DESC;

PRODUCT_NAME         STOCK_QUANTITY
-------------------- --------------
Wireless Mouse                  100
Face wash                        90
Yoga Mat                         80
Bluetooth Speaker                75
Cooking oil                      70
Cotton Shirt                     60
Smart phone                      50
Denim Jeans                      45
Running Shoes                    40
Electric Kettle                  35
Building Blocks                  35
Data Structures                  30
Python programming               25
Office Chair                     15
Study Table                      10

15 rows selected.
```

2. Products generating highest revenue

```
SQL> SELECT PRODUCT_NAME,PRODUCT_ID, SUM(STOCK_QUANTITY)
 AS REVENUE FROM PRODUCT
  2  GROUP BY PRODUCT_NAME, PRODUCT_ID
  3  ORDER BY REVENUE DESC;

PRODUCT_NAME         PRODUCT    REVENUE
-------------------- ------- ----------
Wireless Mouse       P002           100
Face wash            P013            90
Yoga Mat             P015            80
Bluetooth Speaker    P003            75
Cooking oil          P012            70
Cotton Shirt         P004            60
Smart phone          P001            50
Denim Jeans          P005            45
Running Shoes        P009            40
Electric Kettle      P008            35
Building Blocks      P011            35
Data Structures      P006            30
Python programming   P007            25
Office Chair         P010            15
Study Table          P014            10

15 rows selected.
```

3. Least-selling products

```
SQL> SELECT PRODUCT_NAME,PRODUCT_ID,SUM(PRICE*STOCK_QUAN
TITY) AS SELLING_PRICE
  2  FROM PRODUCT GROUP BY PRODUCT_NAME,PRODUCT_ID
  3  ORDER BY SELLING_PRICE ASC;

PRODUCT_NAME         PRODUCT SELLING_PRICE
-------------------- ------- -------------
Data Structures      P006            19500
Python programming   P007            21250
Face wash            P013            31410
Building Blocks      P011            34965
Electric Kettle      P008            45465
Yoga Mat             P015            47920
Cooking oil          P012            48930
Cotton Shirt         P004            53940
Study Table          P014            69990
Denim Jeans          P005            71955
Wireless Mouse       P002            79900
Office Chair         P010            82485
Running Shoes        P009            87960
Bluetooth Speaker    P003           112425
Smart phone          P001          1249950

15 rows selected.
```

4. Products requiring promotion

```
SQL> SELECT OD.PRODUCT_ID, COUNT(*) AS NO_OF_SALES
  2  FROM ORDERS O, ORDER_DETAILS OD
  3  WHERE O.ORDER_ID=OD.ORDER_ID
  4  AND O.ORDER_STATUS='DELIVERED'
  5  GROUP BY OD.PRODUCT_ID
  6  ORDER BY NO_OF_SALES ASC;

PRODUCT NO_OF_SALES
------- -----------
P009              1
P001              1
P007              1
P014              1
P003              1
P005              1
P011              1

7 rows selected.
```

6.Perform Category-wise Sales Analysis:

1.Total sales generated by each category.

```
SQL> SELECT SUM(O.TOTAL_AMOUNT) AS TOTAL_SALE,P.PRODUCT_
NAME
  2  FROM ORDERS O
  3  JOIN ORDER_DETAILS OD ON O.ORDER_ID=OD.ORDER_ID
  4  JOIN PRODUCT P ON P.PRODUCT_ID=OD.PRODUCT_ID
  5  GROUP BY P.PRODUCT_NAME;

TOTAL_SALE PRODUCT_NAME
---------- --------------------
      2750 Data Structures
      3400 Cooking oil
      1600 Yoga Mat
      2500 Smart phone
      2800 Face wash
      1500 Cotton Shirt
      2300 Office Chair
      1900 Building Blocks
      1800 Wireless Mouse
      3200 Bluetooth Speaker
      4200 Denim Jeans
      1200 Electric Kettle
      3600 Python programming
      5100 Running Shoes
      4500 Study Table

15 rows selected.
```

2.Number of Products sold in each category.

```
SQL> SELECT C.CATEGORY_NAME,COUNT(OD.PRODUCT_ID) AS TOTAL_PRODUCT_SOLD
  2  FROM CATEGORY C
  3  JOIN PRODUCT P ON C.CATEGORY_ID=P.CATEGORY_ID
  4  JOIN ORDER_DETAILS OD ON P.PRODUCT_ID=OD.PRODUCT_ID
  5  GROUP BY C.CATEGORY_NAME;

CATEGORY_NAME                                    TOTAL_PRODUCT_SOLD
------------------------------------------------ ------------------
Clothing                                                          2
Home Appliances                                                   1
Books                                                             2
Beauty                                                            1
Electronics                                                       3
Furniture                                                         2
Sports                                                            2
Grocery                                                           1
Toys                                                              1

9 rows selected.
```

3.Highest revenue-generating category.

```
SQL> SELECT SUM(O.TOTAL_AMOUNT) AS TOTAL_SALE, P.PRODUCT
_NAME
  2  FROM ORDERS O
  3  JOIN ORDER_DETAILS OD ON O.ORDER_ID=OD.ORDER_ID
  4  JOIN PRODUCT P ON P.PRODUCT_ID = OD.PRODUCT_ID
  5  GROUP BY P.PRODUCT_NAME
  6  ORDER BY TOTAL_SALE DESC;

TOTAL_SALE PRODUCT_NAME
---------- --------------------
      5100 Running Shoes
      4500 Study Table
      4200 Denim Jeans
      3600 Python programming
      3400 Cooking oil
      3200 Bluetooth Speaker
      2800 Face wash
      2750 Data Structures
      2500 Smart phone
      2300 Office Chair
      1900 Building Blocks
      1800 Wireless Mouse
      1600 Yoga Mat
      1500 Cotton Shirt
      1200 Electric Kettle

15 rows selected.
```

4.Average sales per category.

```
SQL> SELECT AVG(O.TOTAL_AMOUNT) AS AVG_SALE, P.PRODUCT_N
AME
  2  FROM ORDERS O
  3  JOIN ORDER_DETAILS OD ON O.ORDER_ID=OD.ORDER_ID
  4  JOIN PRODUCT P ON P.PRODUCT_ID=OD.PRODUCT_ID
  5  GROUP BY P.PRODUCT_NAME;

  AVG_SALE PRODUCT_NAME
---------- --------------------
      2750 Data Structures
      3400 Cooking oil
      1600 Yoga Mat
      2500 Smart phone
      2800 Face wash
      1500 Cotton Shirt
      2300 Office Chair
      1900 Building Blocks
      1800 Wireless Mouse
      3200 Bluetooth Speaker
      4200 Denim Jeans
      1200 Electric Kettle
      3600 Python programming
      5100 Running Shoes
      4500 Study Table

15 rows selected.
```

7.Generate Business Analytics Reports:

Report 1: Sales Performance Report

- Total sales
- Number of orders
- Average order value
- Highest order value

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES,
  2   COUNT(ORDER_ID) AS NUMBER_OF_ORDERS,
  3  AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE,
  4  MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER_VALUE
  5  FROM ORDERS;

TOTAL_SALES NUMBER_OF_ORDERS AVERAGE_ORDER_VALUE HIGHEST_ORDER_VALUE
----------- ---------------- ------------------- -------------------
      42350               15          2823.33333                5100
```

Report 3: Product Performance Report:

- Product name
- Quantity sold
- Revenue generated

```
SQL> SELECT P.PRODUCT_NAME,
  2  SUM(OD.QUANTITY) AS QUANTITY_SOLD,
  3  SUM(OD.QUANTITY*OD.PRICE) AS REVENUE_GENERATED
  4  FROM PRODUCT P
  5   JOIN ORDER_DETAILS OD ON P.PRODUCT_ID=OD.PRODUCT_I
D
  6  GROUP BY P.PRODUCT_NAME;

PRODUCT_NAME         QUANTITY_SOLD REVENUE_GENERATED
-------------------- ------------- -----------------
Data Structures                  1              2750
Cooking oil                      1              3400
Yoga Mat                         2              1600
Smart phone                      1              2500
Face wash                        1              4500
Cotton Shirt                     3              1500
Office Chair                     1              2300
Building Blocks                  2              1900
Wireless Mouse                   2              1800
Bluetooth Speaker                1              3200
Denim Jeans                      2              4200
Electric Kettle                  1              1200
Python programming               2              3600
Running Shoes                    3              5100
Study Table                      1              4500

15 rows selected.
```

Report 4: Category Analytics Report:

- Category name
- Total products sold
- Total revenue

```
SQL> SELECT C.CATEGORY_NAME,
  2  SUM(OD.QUANTITY) AS TOTAL_PRODUCTS_SOLD,
  3  SUM(OD.QUANTITY*OD.PRICE) AS TOTAL_REVENUE
  4  FROM CATEGORY C
  5  JOIN PRODUCT P ON C.CATEGORY_ID=P.CATEGORY_ID
  6  JOIN ORDER_DETAILS OD ON P.PRODUCT_ID=OD.PRODUCT_ID
  7  GROUP BY C.CATEGORY_NAME;

CATEGORY_NAME                                      TOTAL_PRODUCTS_SOLD TOTAL_REVENUE
-------------------------------------------------- ------------------- -------------
Clothing                                                             5          5700
Home Appliances                                                      1          1200
Books                                                                3          6350
Beauty                                                               1          4500
Electronics                                                          4          7500
Furniture                                                            2          6800
Sports                                                               5          6700
Grocery                                                              1          3400
Toys                                                                 2          1900

9 rows selected.
```